

User Manual

Overview

What Is Tier0?

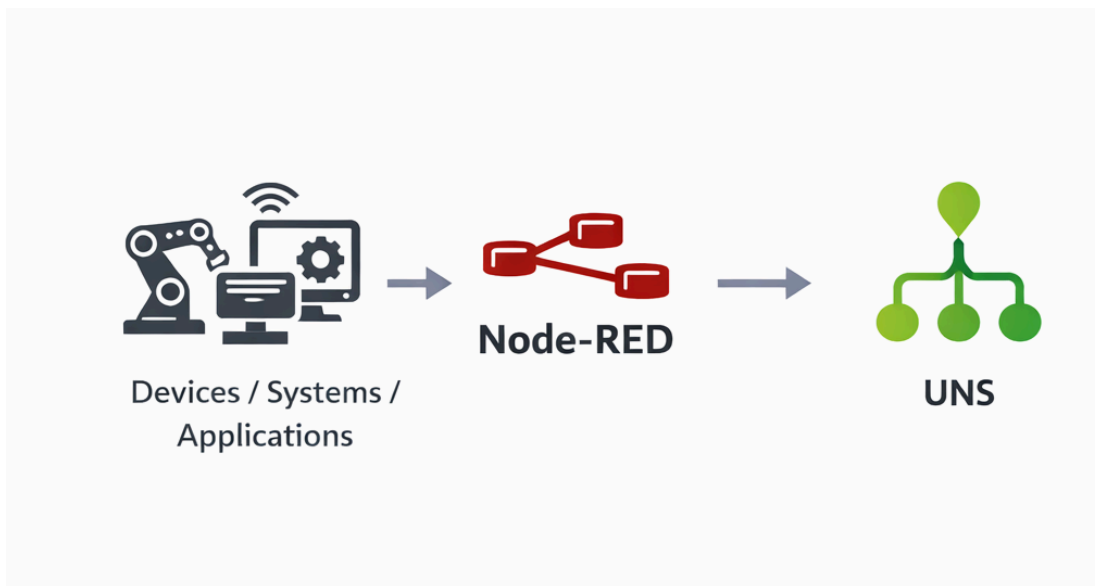
Tier0 is a Unified Namespace (UNS)-based industrial data platform.

It connects data through Node-RED, and organizes it into a unified structure.

How It Works?

Tier0 structures and processes data in a single workflow:

- Node-RED connects and processes data from devices and systems
- Data is published into the Unified Namespace (UNS), structured and stored in Tier0 database



Key Components

Unified Namespace (UNS)

Defines a hierarchical structure for all industrial data.

```
factory/workshopA/line1/cnc01/telemetry/spindleSpeed
```

Namespace

Topic

Template

Label

▼

Q

Please enter name/alias/path

+

↺

factory(1)

workshopA(1)

line1(1)

cnc01(1)

telemetry(1)

Metric(1)

spindleSpeed

factory / workshopA / line1 / cnc01 / telemetry / Metric / spindleSpeed

Import

Export

spindleSpeed

Details

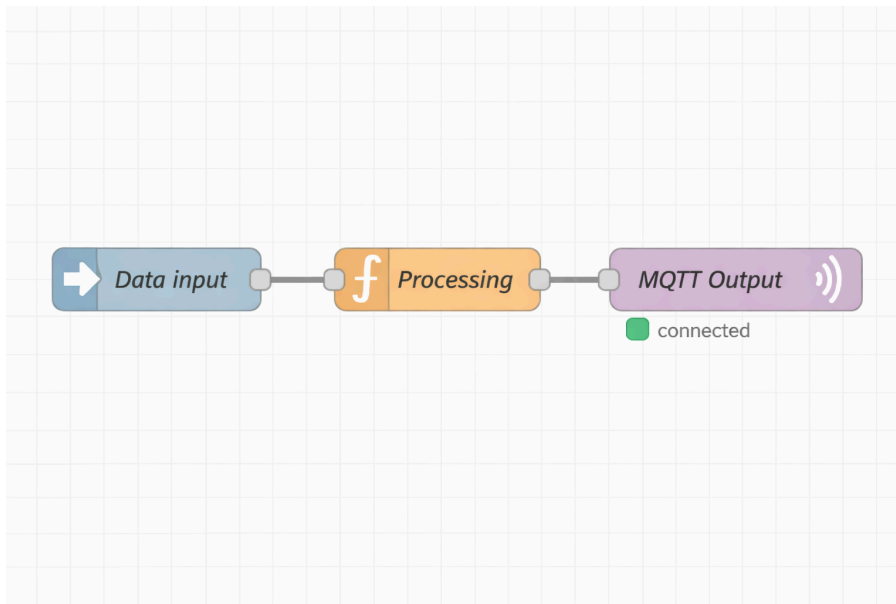
Definition

Attribute	Type	Length	Display Name	Remark
timeStamp	DATE TIME			
speed	INTEGER			
quality	LONG			

Node-RED Integration

The core engine for data connection and processing.

- Connect devices, applications, and systems
- Format and publish data into the namespace



Feature List

This section lists the features of Tier0, and each feature has a corresponding guide in the documentation. You can click the links to learn how to use each feature.

Feature	Description	Remarks
Namespace	Models industrial data into a hierarchical structure (folder-file) for clarity	• Private deployment. • For details on the specifications, see System Requirements.
Source Flow	Connect various data sources via drag-and-drop using Node-RED	

Quick Start Guide

Deploy Tier0

System Requirements

Configuration Type	CPU	Memory	HDD/IOPS
Basic Configuration	4 cores	8 GB	100 GB, 2000 IOPS (30% write)
Recommended Configuration	8 cores	16 GB	1 TB, 2000 IOPS (30% write)

Installing Tier0

Linux

1. Make sure you have the following system and components ready.
 - Ubuntu Server 24.04
 - Docker

Component	Version
Docker Engine - Community	27.4.0
Docker Buildx	v0.19.2
Docker Compose	v2.31.0
Containerd	1.7.24

note

Above system and components have been tested. You can try on other versions if you are interested.

2. Clone the project from Github.

```
git clone https://github.com/FREEZONEX/Tier0-Edge.git
```

3. Go to the folder **Tier0-Edge/deploy** inside the project.

```
cd Tier0-Edge/deploy
```

4. Enter the **.env.default** file and press the **i** key to edit configuration items as needed.

```
vi .env.default
```

Item	Description
VOLUMES_PATH	The storage path for project data.
ENTRANCE_DOMAIN	Tier0 access address. Normally the IP or domain of the server where you install Tier0. ⚠ warning Do not use 127.0.0.1 or localhost, otherwise login and authentication functions will NOT work.
LANGUAGE	Tier0 display language. Chinese (zh-CN) and English (en-US) are supported.

```
# Deployed server specification
# for test
# supported: windows, linux
OS_PLATFORM_TYPE=linux

# Example: 1. 4c8g (4 CPU cores, 8GB RAM), 2. 8c16g (8 CPU cores, 16GB RAM) or higher
OS_RESOURCE_SPEC=1
OS_NAME=Tier0
VOLUMES_PATH=/volumes/supos/data

#(Your IP Address, do not to use 127.0.0.1 or localhost, otherwise the login and authentication functions will not work)
ENTRANCE_PROTOCOL=http
ENTRANCE_DOMAIN=192.168.1.100
ENTRANCE_PORT=8088
ENTRANCE_SSL_PORT=8443

# LANGUAGE setting, supported: en-US, zh-CN
LANGUAGE=en-US
```

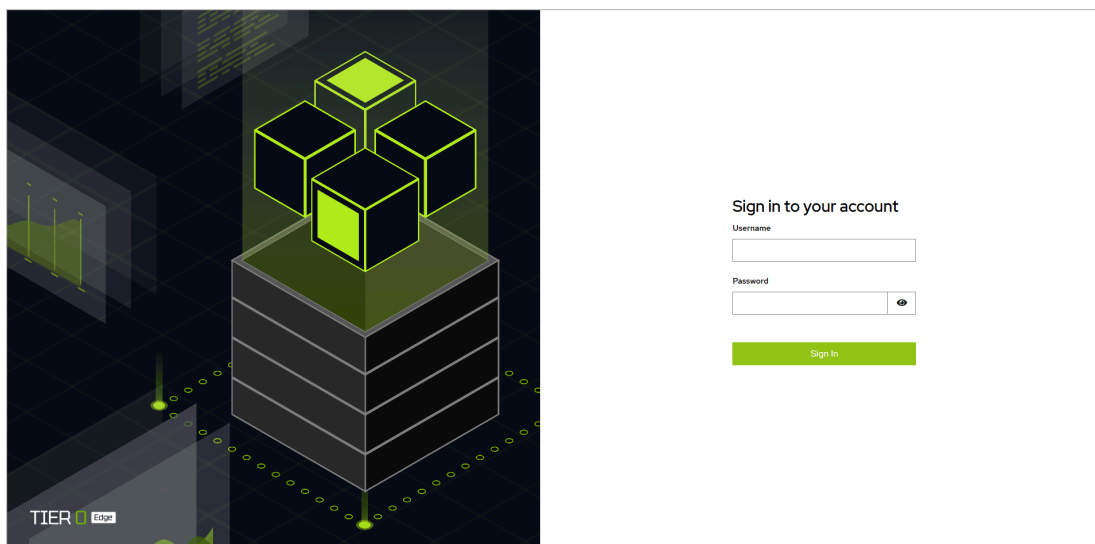
5. Press **ESC** to stop editing, save the file and install Tier0.

```
:wq!
bash bin/install.sh
```

6. Select whether to keep the entered entrance address and whether to use default setup, and then wait for the installation to complete.

```
Available options for ENTRANCE_DOMAIN:
0). Keep current: office.unibutton.com (default)
1). 192.168.31.167
2). 172.17.0.1
3). 172.18.0.1
Select option (0-3), or press Enter to keep current: 0
Keeping current ENTRANCE_DOMAIN: office.unibutton.com
Docker and Docker Compose meet the required versions.
Do you need to customize which services to install? [1] No, use defaults(default) [2] Yes 1
Info: success to generate /root/supOS-CE/bin/init/../../mount/postgresql/init-scripts/create_keycloak_database.sql
Info: success to generate kong_config.yml (auth enabled = true)
Info: loading npm cache complete.
golioth-websocket-datasource/
golioth-websocket-datasource/CHANGELOG.md
golioth-websocket-datasource/gpx_websocket_darwin_amd64
golioth-websocket-datasource/gpx_websocket_darwin_arm64
golioth-websocket-datasource/gpx_websocket_linux_amd64
golioth-websocket-datasource/gpx_websocket_linux_arm
golioth-websocket-datasource/gpx_websocket_linux_arm64
golioth-websocket-datasource/gpx_websocket_windows_amd64.exe
golioth-websocket-datasource/img/
golioth-websocket-datasource/img/assets/
golioth-websocket-datasource/img/assets/golioth-grafana-websockets-plugin-datasource.png
golioth-websocket-datasource/img/assets/grafana-websockets-graphing.png
golioth-websocket-datasource/img/assets/grafana-websockets-plugin-streaming.png
golioth-websocket-datasource/img/logo.svg
golioth-websocket-datasource/LICENSE
golioth-websocket-datasource/MANIFEST.txt
golioth-websocket-datasource/module.js
golioth-websocket-datasource/module.js.LICENSE.txt
golioth-websocket-datasource/module.js.map
```

7. Access Tier0 on a browser and log in.



Windows

info

- Make sure you have installed the latest **Docker Desktop** and **Git** on Windows 10 or 11.
- Docker must be started.

1. Clone the project from Github in **Git Bash**.

```
git clone https://github.com/FREEZONEX/Tier0-Edge.git
```

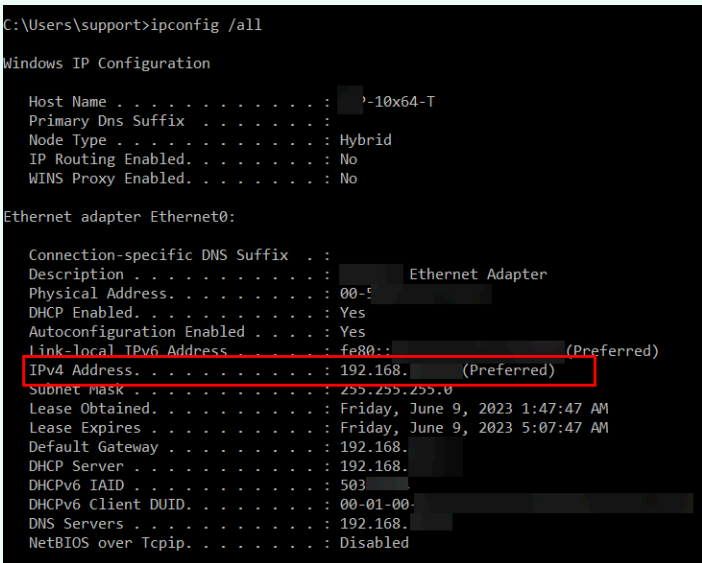
2. Go to the folder **Tier0-Edge/deploy** inside the project.

```
cd Tier0-Edge/deploy
```

3. Edit the following attributes in file **.env.default**.

✓ **tip**

Before editing **.env.default** file, execute **ipconfig** in Git Bash to get the IP of the server.



```
C:\Users\support>ipconfig /all

Windows IP Configuration

Host Name . . . . . : ?-10x64-T
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Ethernet0:

Connection-specific DNS Suffix . :
Description . . . . . : Ethernet Adapter
Physical Address. . . . . : 00-50-56-80-00-00
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::... (Preferred)
IPv4 Address. . . . . : 192.168.1.10 (Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Friday, June 9, 2023 1:47:47 AM
Lease Expires . . . . . : Friday, June 9, 2023 5:07:47 AM
Default Gateway . . . . . : 192.168.1.1
DHCP Server . . . . . : 192.168.1.1
DHCPv6 IAID . . . . . : 503
DHCPv6 Client DUID. . . . . : 00-01-00-00-00-00-00-00-00-00-00-00-00-00-00-00
DNS Servers . . . . . : 192.168.1.1
NetBIOS over Tcpip. . . . . : Disabled
```

- **OS_PLATFORM_TYPE**: The operating system where Tier0 is installed. Change it to **windows**
- **VOLUMES_PATH**: The storage path for project data.
- **ENTRANCE_DOMAIN/ENTRANCE_PORT**: Tier0 access address. Normally the IP or domain of the server where you install Tier0, and make sure the port is not occupied.

```
vi .env.default
```

① **note**

.env.default file is hidden, directly enter the command to open it.


```

MINGW64/d/supOS-CE
# Deployed server specification
# for test
# supported: windows, linux
OS_PLATFORM_TYPE=windows

# Example: 1. 4c8g (4 CPU cores, 8GB RAM), 2. 8c16g (8 CPU cores, 16GB RAM) or higher
OS_RESOURCE_SPEC=2
OS_NAME=supos
VOLUMES_PATH=/c/Users/Free/volumes/supos/data

#(Your IP Address, do not to use 127.0.0.1 or localhost, otherwise the login and authentication functions will not work, )
ENTRANCE_PROTOCOL=http
ENTRANCE_DOMAIN=192
ENTRANCE_PORT=8

# LANGUAGE setting, supported: en-US, zh-CN
LANGUAGE=en-US

# Operating system version
OS_VERSION=V1.00.00.05-C

# MQTT TCP port
OS_MQTT_TCP_PORT=1883

# MQTT WebSocket TLS port
OS_MQTT_WEBSOCKET_TLS_PORT=8084

# Login path for the system
OS_LOGIN_PATH=/supos-login

# Enable or disable authentication (true/false)
OS_AUTH_ENABLE=true

# Type of large language model used
OS_LLM_TYPE=ollama

# Model and Key for oepnai (Enter your api key, can be empty)
OPENAI_API_MODEL=gpt-4o
OPENAI_API_KEY=

# Below are the environment variables required for using the built-in supos MCP with the MCP Client (if applicable, can be empty)
# Langsmith API key
LANGSMITH_API_KEY=
# Agent service address (default: http://localhost:8123)
AGENT_DEPLOYMENT_URL=http://mcpclient:8123
# open API key for supos, used for external access to the system
SUPOS_API_KEY=4174348a-9222-4e81-b33e-5d72d2fd7f1e
# The address of the MQTT server of supos that needs to be accessed
SUPOS_MQTT_URL=tcp://emqx:1883/mqtt

#Variables from older versions, will be deleted for next iteration, please do not modify
MULTIPLE_TOPIC=false

#-----OAUTH CONFIG-----
OAUTH_REALM=supos
.env [unix] (10:22 20/05/2025)
".env" [unix] 69L, 1988B

```

4. Save the file and start installing Tier0.

```

:wq!
bash bin/install.sh

```

5. Select the default configurations and wait for the installation to complete.

```
MINGW64:/d/supOS-CE
Free@DESKTOP-JK7K19R MINGW64 /d/supOS-CE (main)
$ bash bin/startup.sh
Choose VOLUMES_PATH: (Press Enter for default: [/c/Users/Free/volumes/supos/data])
Choose IP address for ENTRANCE_DOMAIN (Press Enter for default: [192.168.1.1]):
Docker and Docker Compose meet the required versions.
Do you need to customize which services to install? [1] No, use defaults(default) [2] Yes 1
Info: success to generate /d/supOS-CE/bin/init/../../mount/postgresql/init-scripts/create_keycloak_database.s
ql
Info: success to generate kong_config.yml (auth enabled = true)
Info: loading npm cache complete.
golioth-websocket-datasource/
golioth-websocket-datasource/CHANGELOG.md
golioth-websocket-datasource/gpx_websocket_darwin_amd64
golioth-websocket-datasource/gpx_websocket_darwin_arm64
golioth-websocket-datasource/gpx_websocket_linux_amd64
golioth-websocket-datasource/gpx_websocket_linux_arm
golioth-websocket-datasource/gpx_websocket_linux_arm64
golioth-websocket-datasource/gpx_websocket_windows_amd64.exe
golioth-websocket-datasource/img/
golioth-websocket-datasource/img/assets/
golioth-websocket-datasource/img/assets/golioth-grafana-websockets-plugin-datasource.png
golioth-websocket-datasource/img/assets/grafana-websockets-graphing.png
golioth-websocket-datasource/img/assets/grafana-websockets-plugin-streaming.png
golioth-websocket-datasource/img/logo.svg
golioth-websocket-datasource/LICENSE
golioth-websocket-datasource/MANIFEST.txt
golioth-websocket-datasource/module.js
golioth-websocket-datasource/module.js.LICENSE.txt
golioth-websocket-datasource/module.js.map
golioth-websocket-datasource/plugin.json
golioth-websocket-datasource/README.md
Info: success to create folder: /c/Users/Free/volumes/supos/data
time="2025-05-20T10:22:57+08:00" level=warning msg="D:\\supOS-CE\\docker-compose-8c16g.yml: the attribute 've
rsion' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 14/14
  ✓ Network supos_default_network Created 0.0s
  ✓ Container postgresql Healthy 62.0s
  ✓ Container eventflow Started 0.8s
  ✓ Container chat2db Started 0.8s
  ✓ Container konga Started 0.4s
  ✓ Container emqx Started 1.0s
  ✓ Container portainer Started 0.6s
  ✓ Container frontend Started 0.6s
  ✓ Container hasura Started 61.3s
  ✓ Container nodered Started 1.2s
  ✓ Container keycloak Healthy 81.9s
  ✓ Container grafana Started 61.4s
  ✓ Container backend Started 82.1s
  ✓ Container kong Started 82.3s
>> start to init nodered modules ...
added 55 packages in 4s
28 packages are looking for funding
  run 'npm fund' for details
npm warn deprecated node-opcua-client-crawler@2.127.1: true
```

6. Access Tier0 on a browser and log in.

Uninstalling Tier0

1. Access the path **Tier0-Edge/deploy/bin**.
2. Execute the script **uninstall.sh**.

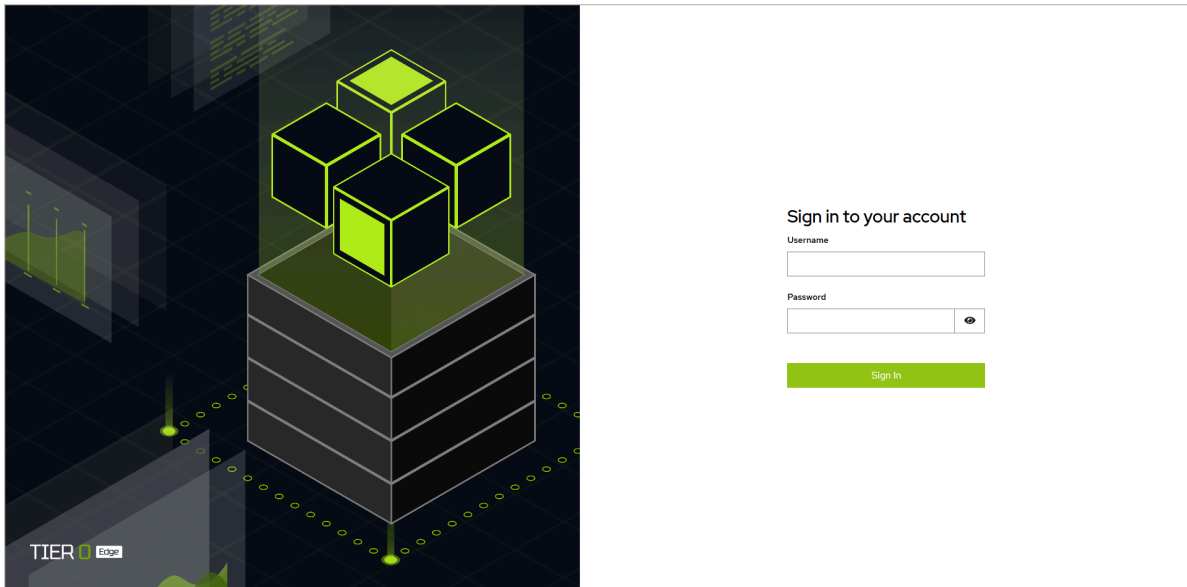
```
bash uninstall.sh
```

① info

- **clean-all.sh** will wipe out all project data and uninstall Tier0.
- Images will not be deleted.
- If you want to install Tier0 again, make sure you have pulled all images with the newest versions (based on image tags).

Login

1. Access Tier0 with the set domain and port on a browser.
2. On the login page, enter username **tier0** and password **tier0**.



3. Click **Sign In**.

How-to Guide

Build Data Models

Tier0 uses UNS (Unified Namespace) to organize data into a clear tree structure. Each path and topic in the model automatically generates an MQTT topic, making it easy to get real-time data.

Additional Parameter Definition

- **Path-Additional Settings**

Item	Description
Extended Attribute	Add extended attributes for the path. Eg. unit.
Template	Select a template to inherit attributes from it. See Building Template .
Generate Template	Click  to add an attribute to the path, at the same time you can select to generate a same template for future use.

- **Topic-Topic Type**

Type	Description
State	Works as an API to receive system state data.
Action	Works as an API to execute CRUD commands.
Metric	Real-time data. For example, equipment temperature.

- **Topic-Attribute Generation Method**

Attribute Generation Method	Description
Pre-defined	Customize attributes.
Template	Inherit attributes from the selected template.
Auto-Parsed	Use JSON text to generate attributes with the same structure.

- **Topic-Linked Operation**

Item	Description
Enable History	Enable historical data storage for the topic.

How to Build a Data Model

Based on simple folder-file structure, you can define the data hierarchy to a tree map.

Building Models Manually

① note

factory/equipment/CNC will be used as an example, in which `factory` and `equipment` are paths and `CNC` is a topic.

1. Log in to Tier0, landed on the **Namespace** page.
2. Under **Topic**, click **+ > New Path** to add a path (e.g. `factory`).

New Path

×

Namespace ?

* Name

▼ Additional Settings

* Alias ?

__d45036a7cb0e4c258979

Display Name

Path Description

Extended Attribute

+

Template

Custom

▼

Attribute ?

+

Save

3. Enter the path information, set the alias as needed, and then click **Save**.
4. Select **equipment**, and then click **+** > **New Topic** to add a topic (e.g. CNC) under it.

New Topic

×

Namespace ?

equipment/

* Name

Topic Type ?

☒ State
☐ Action
☐ Metric

Attribute

Generation ?

☒ Custom
☐ Template
☐ Auto Parsing

Method

Attribute ?

Name

Type ▼

Length

Display N...

Remark(...)

⊖

⊕

▼ Additional Settings

* Alias ?

__be82973cc1f7449ca148

Display Name

Topic Description

Label ?

▼

Extended Attribute

⊕

Optional Behaviors

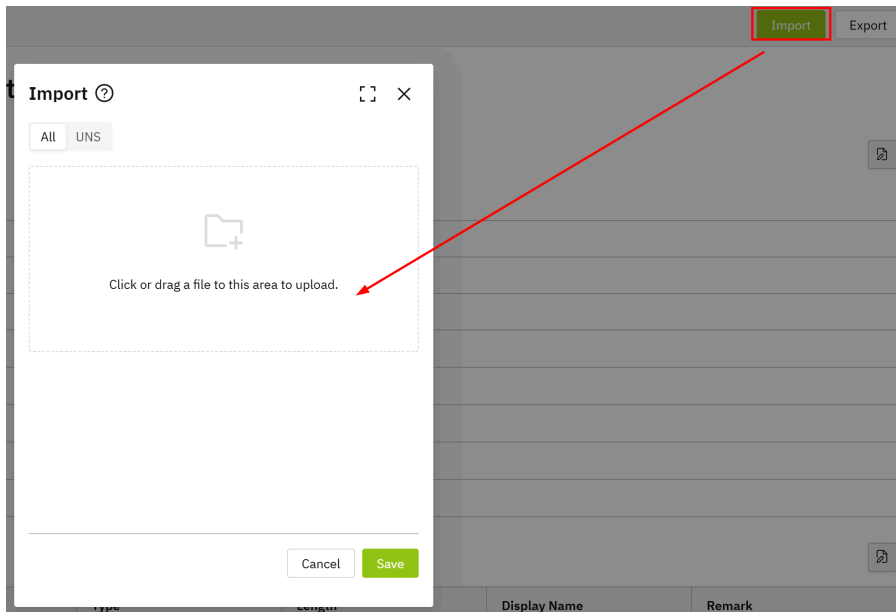
☐ Enable History ?

Save

5. Enter the information of the topic, and then click **Save**.

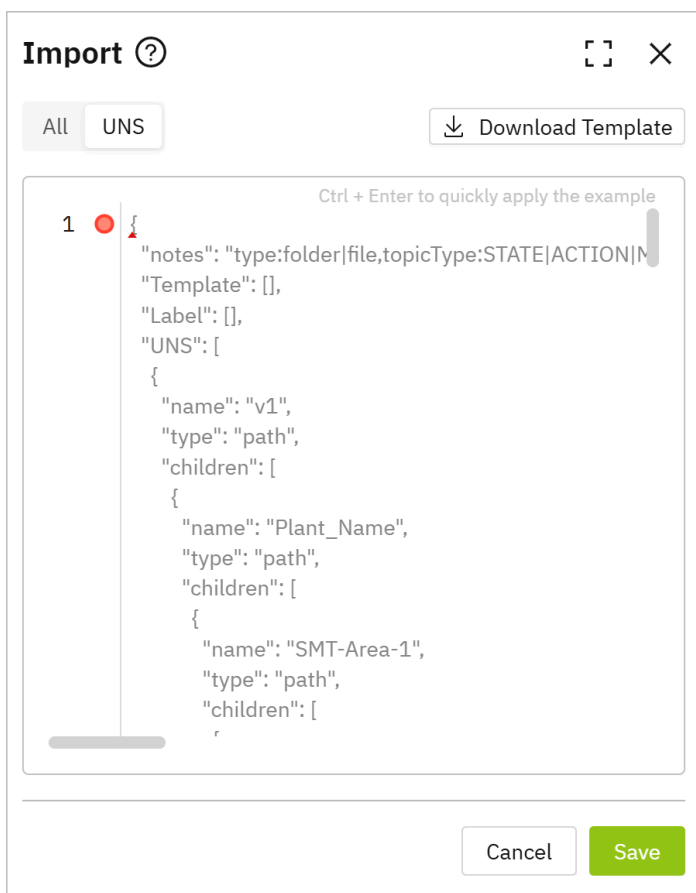
Importing Models

1. Log in to Tier0, landed on the **Namespace** page.
2. Click **Import** at the upper-right corner.



3. Import JSON to create models.

- Click **UNS**, and directly enter JSON.

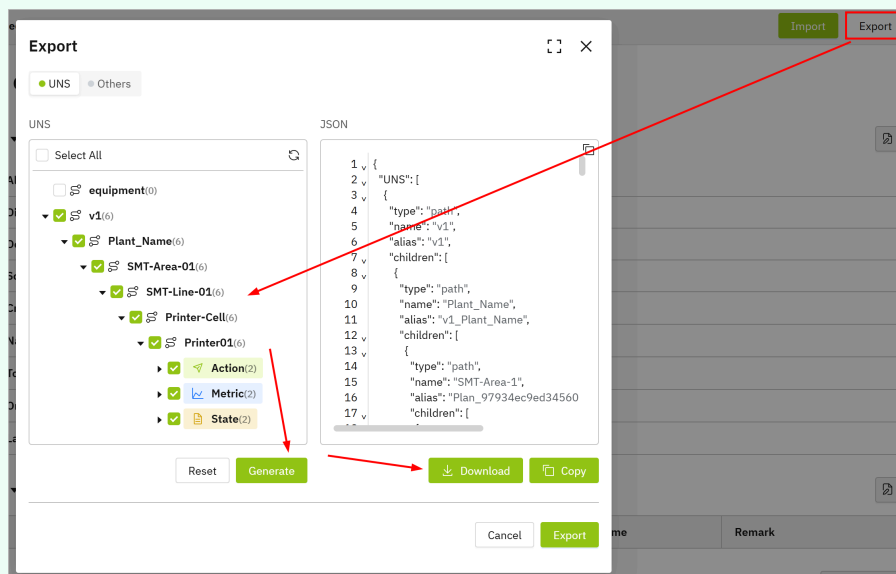


- Click **Download Template** at the **UNS** tab, enter the model content and upload it at the **All** tab.


```
{
  "notes": "type:folder|file,topicType:STATE|ACTION|METRIC,dataType:TEMPLATE_TYPE|TIME_SEQUENCE_TYPE|RELATION",
  "Label": [{"name": "debug"}, {"name": "dev"}],
  "Template": [],
  "UNS": [
    {
      "name": "v1",
      "type": "path",
      "children": [
        {
          "name": "Plant_Name",
          "type": "path",
          "children": [
            {
              "name": "SMT-Area-1",
              "type": "path",
              "children": [
                {
                  "name": "SMT-Line-1",
                  "type": "path",
                  "children": [
                    {
                      "name": "Printer-Cell",
                      "type": "path",
                      "children": [
                        {
                          "name": "Printer01",
                          "type": "path",
                          "children": [
                            {
                              "name": "State",
                              "type": "path",
                              "topicType": "STATE"
                            }
                          ]
                        }
                      ]
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

✓ tip

You can manually add a path and topic, export it and use it as an example for import.



4. Click **Save**.

Optional Features

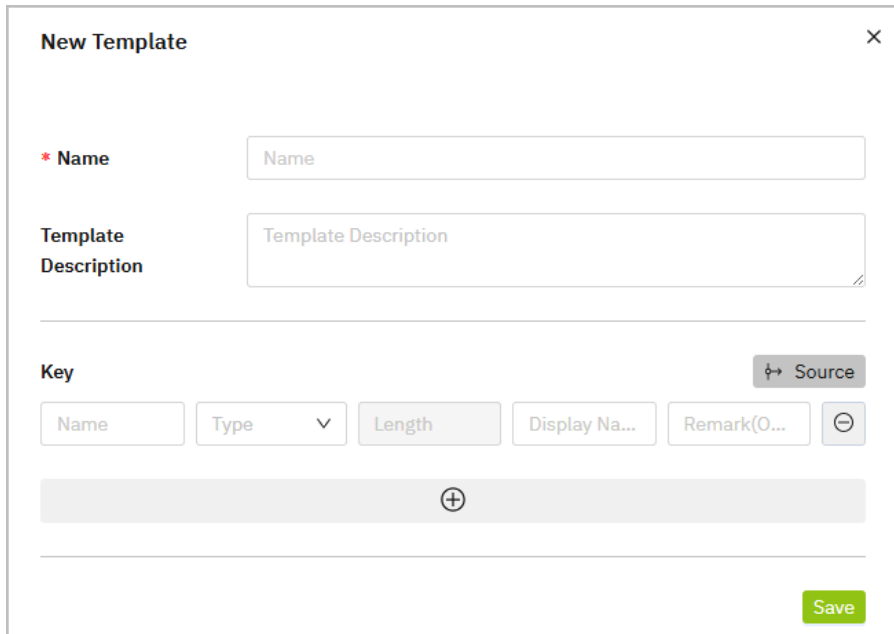
Building Template

1. On the **Namespace** page, click **Template**.

2. Click  , and then enter the information of the template.

✓ **tip**

Click **Source** to select added models and inherit their attributes.



The 'New Template' dialog box contains the following fields and controls:

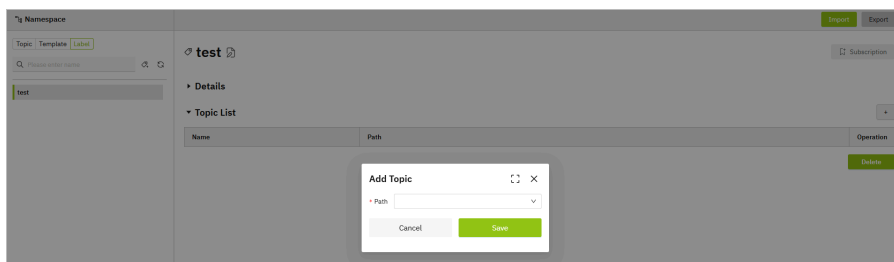
- Name:** A text input field with a red asterisk indicating it is required.
- Template Description:** A larger text area for describing the template.
- Key:** A section containing a 'Source' button with a key icon and a list of key attributes: 'Name', 'Type' (with a dropdown arrow), 'Length', 'Display Na...', and 'Remark(O...'. A minus sign button is also present.
- Save:** A green button at the bottom right.

3. Click **Save**.

Creating Label

Labels are used for categorizing data models.

1. On the **Namespace** page, click **Label**.
2. Click  , and then enter the label name.
3. Click **Save**, and then click  to add topics under the label.



The screenshot shows the 'Namespace' page with a 'test' label selected. An 'Add Topic' dialog box is open, allowing the user to enter a 'Path' and click 'Save'.

4. Click **Save**.

Exporting Data

Tier0 can export UNS models and other data, such as source flows, event flows, and data dashboards.

1. On the **Namespace** page, click **Export** at the upper-right corner.
2. Click the **Others** tab and select the corresponding data type.

Export

● UNS ● Others

Source Flow

Please select

Event Flow

Please select

Dashboard

Please select

Cancel

Export

3. Click **Export**.

Connect Data Sources

How to Add Data Sources

Once the data model is built under **Namespace**, you need to connect real data to the model for subsequent processing and use.

Adding Data Flow Manually

1. Log in to Tier0, go to **UNS > Source Flow**.

Source Flow					1	2	3
Please enter name or description					Search	New Source Flow	
Name	Flow Template	Description	Creation Time	Creator	Operation		
factory/equipment/Metric/CNC2	node-red	Running	17:38:12 25/11/2025	supos			
test	node-red	Draft	17:12:23 25/11/2025	supos			
Total 2 Items					1	18 / page	

Index	Item	Description
1	Search	Search for existing data flows.
	New Source Flow	Add a data flow.
2	Flow List	Displays flow information. Click the flow name to start editing the data flow.
3	Display	Switch flow display between blocks and list.
4	Flow Operations	Copy / Edit / Delete the data flow.

2. Click **New Source Flow**, enter a name and click **Save**.
 3. Click the flow you just added, and drag in nodes according to your data source type (e.g. modbus).
 4. Drag in an mqtt out node, and finish its configurations.
1. Add the internal broker via emqx:1883 .

Edit mqtt out node > Add new mqtt-broker config node

Cancel Add

Properties

Name

Connection Security Messages

Server Port

☒ Connect automatically

☐ Use TLS

Protocol

Client ID

Keep Alive

Session ☒ Use clean session

2. Set its subscription topic to the data model in **Namespace**.

For example, the data model structure is factory/equipment/cnc , the topic will be factory/equipment/cnc .

Edit mqtt out node

Delete Cancel Done

Properties

Server emqx:1883

Topic factory/equipment/cnc

QoS Retain

Name Name

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

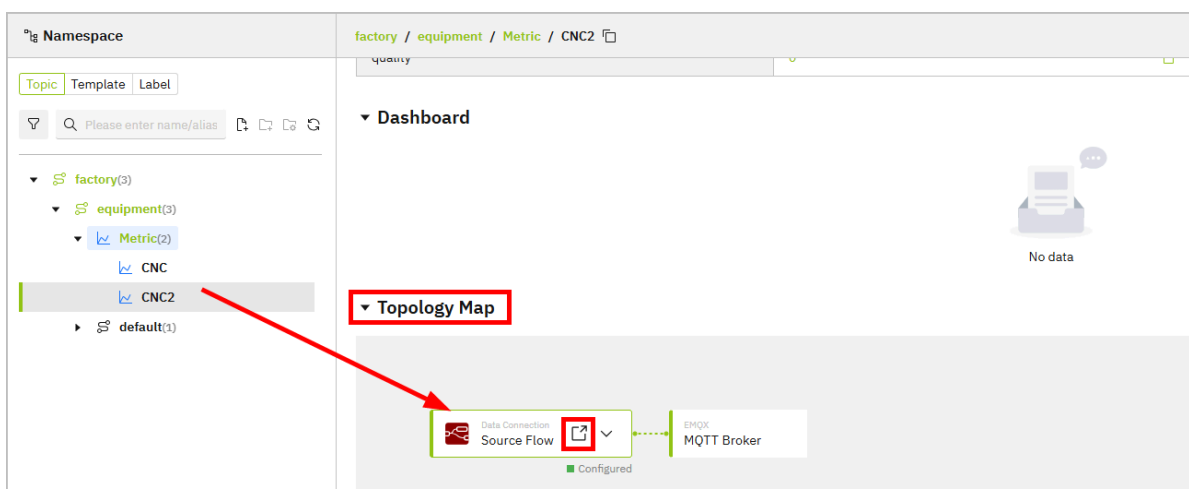
- Click **Deploy** at the upper-right corner, trigger the flow and the data will be sent to corresponding model under **Namespace**.

Configuring Data Source through Generated Flow

note

Data flow will only be automatically generated when you select **Mock Data** while creating topics.

- Log in to Tier0, landed on the **Namespace** page, and under the **Topic** tab, select a file.
- Scroll down to **Topology Map**, click the icon on **Source Flow** to redirect to the generated data flow under **Source Flow**.



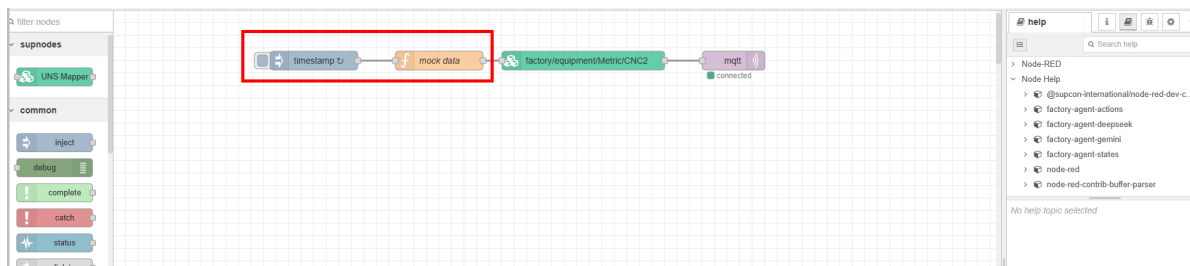
✓ tip

Click  to change the bound data flow.

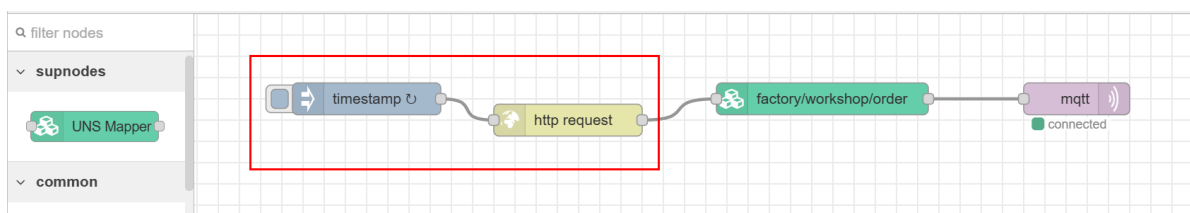
3. Change the data source of the generated flow.

❗ info

Make sure the returned data of the source node is **JSON object**. Currently, Tier0 can only parse this data type.



4. Delete the `mock data` function node and connect the source node to the rest of the flow.



5. Click **Deploy** at the upper-right corner, and then trigger the flow.

6. Go back to **Namespace**, check whether the data is received.

▼ Payload		
Key Name	Value	Latest Update
actualRuntime	1285	11:24:55 16/05/2025
plannedRuntime	1440	11:24:55 16/05/2025

Common Data Sources

Connecting OPC UA

Read Data from OPC UA Server

1. In **Source Flow**, drag an **inject** node.
2. Double-click the node, and then enter the **NodeId** from your OPC UA server.

Dialog box titled "Edit inject node". It has buttons for "Delete", "Cancel", and "Done". The "Properties" tab is selected. The "Name" field is empty. The "msg. payload" field is set to "milliseconds since epoch". The "msg. topic" field is highlighted with a red box and set to "ns=2;i=2".

3. Drag an **OpCua-Client** node, and then double-click it.
4. Click **+** next to **Endpoint** to enter the information of your OPC UA server.

Dialog box titled "Edit OpCua-Client node > Add new OpCua-Endpoint config node". It has buttons for "Cancel" and "Add". The "Properties" tab is selected. The "Endpoint" field is set to "opc.tcp://192.168.1.104:40". The "SecurityPolicy" field is set to "None". The "SecurityMode" field is set to "None". The "Anonymous" checkbox is checked. The "use credentials" and "user certificate" checkboxes are unchecked.

5. Click **Add**, and set **Action** to **READ**.
6. Leave everything else as default, and then click **Done**.

Edit OpcUa-Client node

Delete Cancel Done

Properties

Endpoint: opc.tcp://192.168.1.100:4840

Action: READ

Certificate: None, use generated self-signed certificate

Local certificate file, fixed path: Path is not used anymore! Example: selfSigned.cer

Local private key file, fixed path: Path is not used anymore! Example: private_key.pem

Use transport settings: ☐

Max ChunkCount: 1

Max MessageSize: 8192

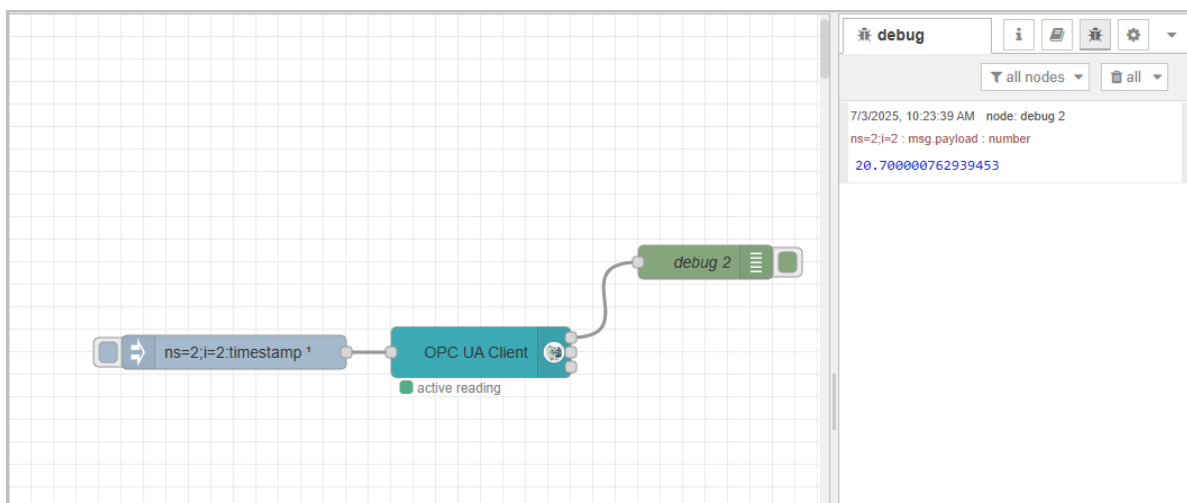
Receive BufferSize: 8192

Send BufferSize: 8192

Set statuscode and timestamp: ☒

Keep session alive: ☒

7. Connect the **inject** node to the **OpcUa-Client** node, and then connect a debug node at the end.
8. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



Subscribing Data from OPC UA Server

1. In **Source Flow**, drag an **inject** node.
2. Double-click the node, and then enter the **NodeId** from your OPC UA server.

- msg.payload:

```
[
  {
    "nodeId": "ns=2;i=1001",
    "datatype": "Int32"
  },
  {
    "nodeId": "ns=2;i=1002",
    "datatype": "Int32"
  }
]
```

- msg.topic: multiple

Dialog box: Edit inject node

Buttons: Delete, Cancel, Done

Properties:

- Name: [Text Field]
- msg.payload: [{"nodeId": "ns=2;i=1001", "datatype": "Int32"}, {"nodeId": "ns=2;i=1002", "datatype": "Int32"}]
- msg.topic: multiple

3. Drag an **OpCua-Client** node, and then double-click it.

4. Click **+** next to **Endpoint** to enter the information of your OPC UA server.

Dialog box: Edit OpCua-Client node > Add new OpCua-Endpoint config node

Buttons: Cancel, Add

Properties:

- Endpoint: opc.tcp://192.168.1.100:4840
- SecurityPolicy: None
- SecurityMode: None
- Anonymous: ☒
- use credentials: ☐
- user certificate: ☐

5. Click **Add**, and set **Action** to **SUBSCRIBE**.

6. Set the subscription interval, and leave everything else as default, and then click **Done**.

Edit OpcUa-Client node

Delete Cancel Done

Properties

Endpoint: opc.tcp://192.168.1.104:4840

Action: SUBSCRIBE

Interval: 2 second(s)

Certificate: None, use generated self-signed certificate

Local certificate file, fixed path: Path is not used anymore! Example: selfSigned.cer

Local private key file, fixed path: Path is not used anymore! Example: private_key.pem

Use transport settings: ☐

Max ChunkCount: 1

Max MessageSize: 8192

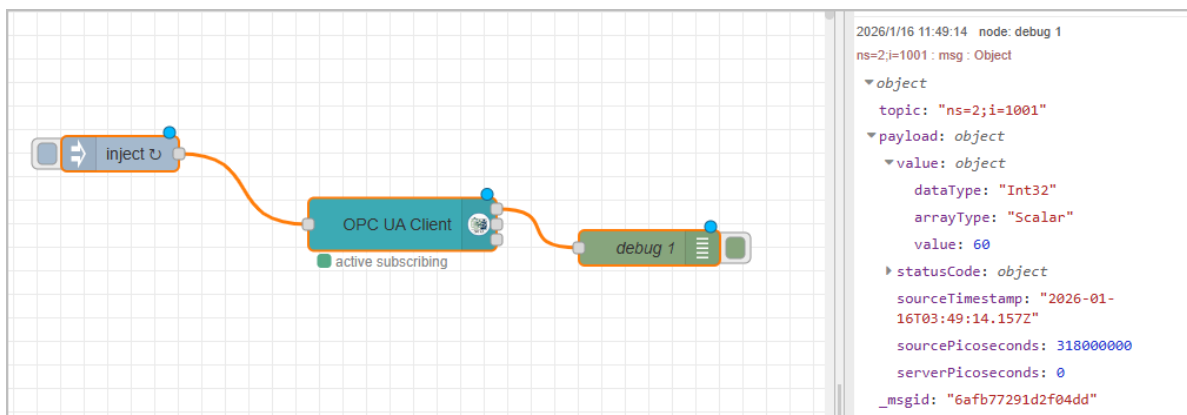
Receive BufferSize: 8192

Send BufferSize: 8192

Set statusCode and timestamp: ☐

7. Connect the **inject** node to the **OpcUa-Client** node, and then connect a debug node at the end.

8. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



Connecting OPC DA

1. In **Source Flow**, drag an **inject** node.
2. Drag an **opcda-read** node, and then double-click it.
3. Click **+** next to **Server** to enter the information of your OPC DA server.

Edit opcda-read node > Add new opcda-server config node

Cancel Add

Properties

Name: opcda

Address: 192.168.31.142

Domain: "

User Name: admin

Password:

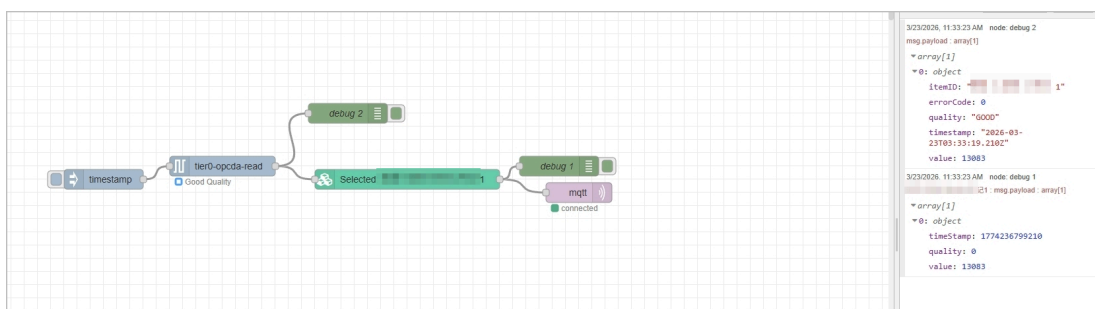
ClsId: 7BC0CC8E-482C-47CA-ABDC-0FE7F9C6E729

Timeout: 5000 ms

Browse Export

Parameter	Description
Address	OPC DA server IP address.
Domain	The domain name of your OPC DA server.
User Name/Password	The username and password you use to log in to the DA server.
ClsId	The application ID of the DA connection service you use. For example, KepServer.

- Click **Browse** to see the connected channels.
- Click **Add**, and then under **Items**, add the items needed.
- Connect the **inject** node to the **opcda-read** node, and then connect a debug node at the end.
- Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



Connecting Modbus

1. In **Source Flow**, drag a **Modbus Read** node.
2. Double-click the node, and then click **+** next to **Server** to enter the information of your modbus server.

Edit Modbus-Read node > Add new modbus-client config node

Cancel Add

Properties

Settings Queues Optionals

Name modbus

Type TCP

Host 192.168.1.100

Port 502

TCP Type DEFAULT

Unit-Id 1

Timeout (ms) 1000

Reconnect on timeout ☒

Reconnect timeout (ms) 2000

3. Click **Add**, and then enter the information of the data to be connected.

Edit Modbus-Read node

Delete Cancel Done

Properties

Settings Optionals

Name modbus

Topic Topic

Unit-Id

FC FC 3: Read Holding Registers

Address 0

Quantity 1

Poll Rate 10 second(s)

Delay to activate input ☐

Server modbus

Parameter	Description
Name	The display name of the node (for visual identification only).
Topic	Optional topic string passed to output messages.
Unit-Id	The Modbus slave ID of the target device (typically 1–247).
FC	Function code that specifies the Modbus action, e.g., FC 3 = Read Holding Registers.
Address	The starting register address to read from (usually zero-based).
Quantity	The number of consecutive registers to read.
Poll Rate	How often the node polls the Modbus device (e.g., every 10 seconds).
Delay to activate input	Optional delay before activating the input signal for polling.
Server	Reference to a configured Modbus server (IP, port, protocol, etc.).

4. Click **Done**.
5. Connect a debug node at the end.
6. Click **Deploy** at the upper-right corner, and then click **Modbus Read** node to trigger the flow.

The screenshot shows a Node-RED workspace with a 'modbus' node connected to a 'debug 1' node. The 'modbus' node has a green status indicator and is labeled 'active (10 sec.)'. The right sidebar displays the debug console for 'debug 1', showing four log entries:

```

7/4/2025, 11:58:23 AM node: debug 1
polling : msg.payload : array[1]
  array[1]
    0: 1

7/4/2025, 11:58:27 AM node: debug 1
polling : msg.payload : array[1]
  array[1]
    0: 1

7/4/2025, 11:58:37 AM node: debug 1
polling : msg.payload : array[1]
  [ 1 ]

7/4/2025, 11:58:47 AM node: debug 1
polling : msg.payload : array[1]
  [ 1 ]

```

Connecting MQTT

1. In **Source Flow**, drag an **mqtt in** node.
2. Double-click the node, and then click  next to **Server**.

Edit mqtt in node > Add new mqtt-broker config node

Cancel Add

Properties

Name

Connection Security Messages

Server e.g. localhost Port 1883

☒ Connect automatically

☐ Use TLS

Protocol MQTT V3.1.1

Client ID Leave blank for auto generated

Keep Alive 60

Session ☒ Use clean session

Parameter	Description
Name	Optional name for the MQTT broker configuration (for internal reference).
Server	The hostname or IP address of the MQTT broker (e.g., <code>localhost</code> or <code>broker.hivemq.com</code>).
Port	The port used to connect to the MQTT broker (default is <code>1883</code> for non-TLS, <code>8883</code> for TLS).
Connect automatically	If enabled, Node-RED will try to connect to the broker automatically when the flow starts.
Use TLS	Enable this option if the broker requires a secure TLS/SSL connection.
Protocol	The MQTT protocol version to use for connection (e.g., MQTT v3.1.1 or v5.0).
Client ID	Optional client ID. If blank, a unique one will be auto-generated. Must be unique per client.
Keep Alive	The maximum period (in seconds) between communications with the broker. Helps detect dropped connections.
Use clean session	If enabled, no persistent session is maintained with the broker; otherwise, session state (like subscriptions) is retained.
Username (Security tab)	Username for authenticating with the MQTT broker, if required.
Password (Security tab)	Password corresponding to the username for broker authentication. Stored securely.

3. Click **Add**, and then enter the topic to be subscribed.

4. Click **Done**.

Edit mqtt in node

Delete Cancel Done

Properties

Server: emqx

Action: Subscribe to single topic

Topic: factory/RCS/AGV

QoS: 2

Output: auto-detect (parsed JSON object, string or buff)

Name: Name

5. Connect a debug node at the end.

6. Click **Deploy** at the upper-right corner, and then once the topic receives data, the flow will be triggered.

The screenshot shows the Node-RED web interface. On the left, a flow is built on a grid. It starts with a purple MQTT node labeled 'factory/RCS/AGV' with a green 'connected' indicator. A line connects it to an orange 'debug 1' node. On the right, the 'debug' console is open, showing two log entries from 'node: debug 1' at 2:24:45 PM and 2:24:55 PM. Both entries show the message 'factory/RCS/AGV : msg.payload : Object'. The first entry shows a simple object with 'no: 66', 'position', 'state', and 'battery'. The second entry shows a more complex object with 'no: 60', 'position', 'state', and 'battery'.

```

7/4/2025, 2:24:45 PM node: debug 1
factory/RCS/AGV : msg.payload : Object
  object
    no: 66
    position: "3fcLKwHDAvthuZLEwW8V"
    state: "DqGG213IuzMDT73GnVS3"
    battery: "62.91"

7/4/2025, 2:24:55 PM node: debug 1
factory/RCS/AGV : msg.payload : Object
  { no: 60, position: "6CcOvOZfJM8zaCCJA15e", state: "zmP15enJhhK064mK16wV", battery: "63.99" }

```

Connecting File

info

Before processing files in **Source Flow**, you need to put the file in to the corresponding docker.

```
scp C:\Users\you\Desktop\myfile.txt username@server IP:/home/username/  
//copy the file from local to server  
docker cp /root/config.json <docker name or ID>:/app/config.json  
//copy the file from server to docker
```

✓ general docker commands

Command	Description
docker ps	Lists all running Docker containers.
docker ps grep [container]	Filters running containers based on the specified docker information.
docker exec -it [container] /bin/sh	Starts a shell session inside a running container using /bin/sh .
mkdir [directory]	Creates a new directory with the specified name inside a container.

Connecting a Text File

1. In **Source Flow**, drag an **inject** node.
2. Drag a **read file** node, and then double-click it.
3. Enter the file path and output configuration.

Edit read file node

Delete

Cancel

Done

Properties

Filename

path /file-test.txt

Output

a single utf8 string

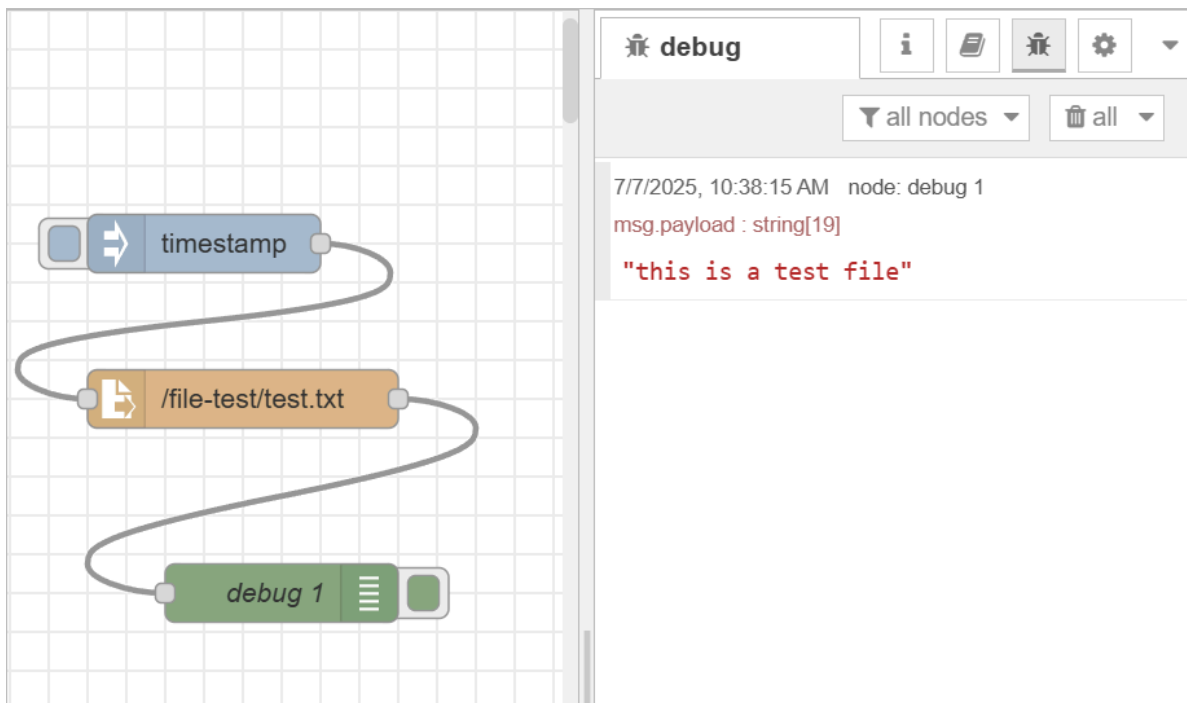
Encoding

default

Name

Tip: The filename should be an absolute path, otherwise it will be relative to the working directory of the Node-RED process.

4. Click **Done**, and then connect a debug node at the end.
5. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.

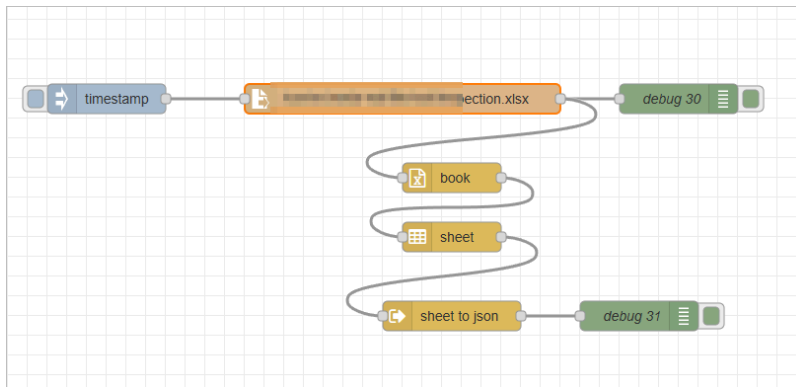


Connecting an Excel File

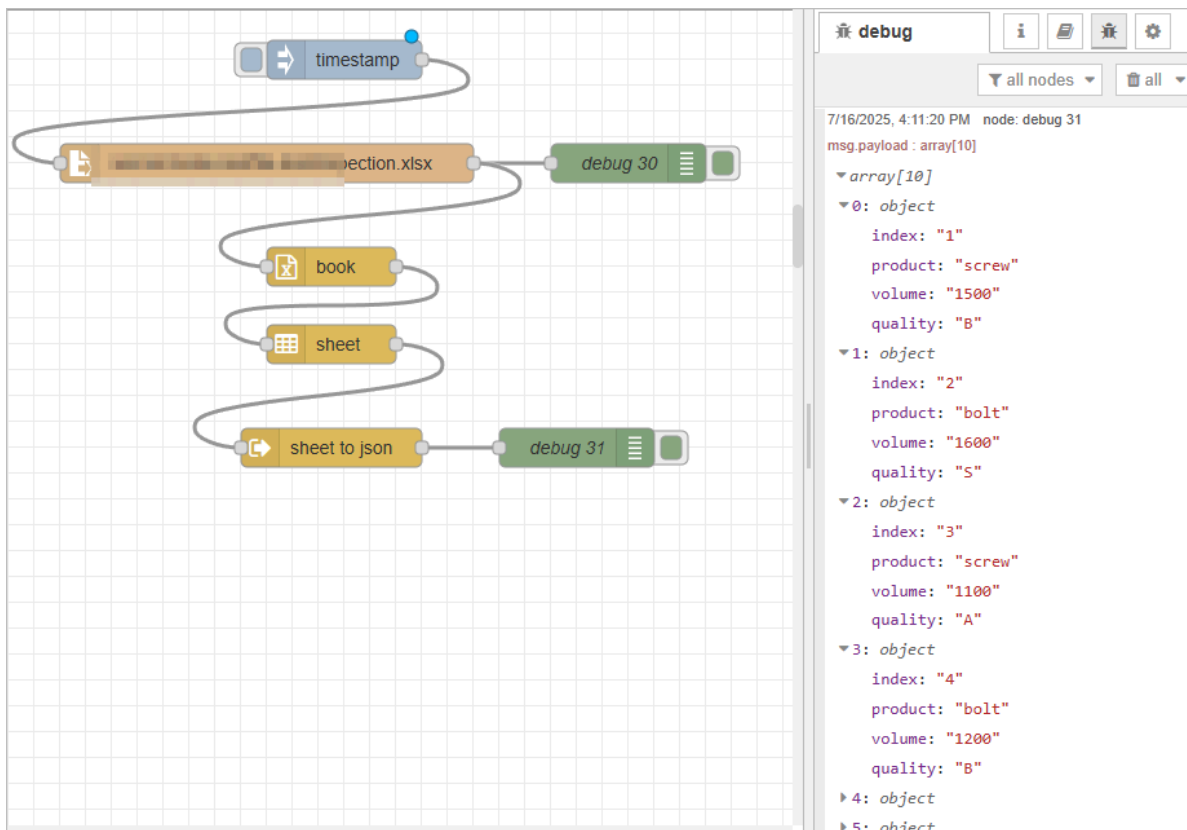
1. In **Source Flow**, drag an **inject** node.
2. Drag a **read file** node, and then double-click it.
3. Enter the file path and output configuration.

The screenshot shows the 'Edit read file node' dialog box. It has a title bar 'Edit read file node' and buttons for 'Delete', 'Cancel', and 'Done'. Below the buttons is a 'Properties' section with a gear icon and three sub-sections: 'Filename' with a dropdown menu showing 'path /...', 'Output' with a dropdown menu showing 'a single Buffer object', and 'Name' with an empty text input field. A yellow tip box at the bottom states: 'Tip: The filename should be an absolute path, otherwise it will be relative to the working directory of the Node-RED process.'

4. Click **Done**, and then connect a **book**, a **sheet** and a **sheet to json** node by order to convert table data to json.



5. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



Connecting RestAPI

1. In **Source Flow**, drag an **inject** node.
2. Drag an **http request** node, and then double-click it.
3. Enter the API information.

✓ tip

Set the **Return to a parsed JSON object** for Tier0 namespace to parse.

Edit http request node

Delete Cancel Done

Properties

Method GET

URL http://

Payload Ignore

☐ Enable secure (SSL/TLS) connection

☐ Use authentication

☐ Enable connection keep-alive

☐ Use proxy

☐ Only send non-2xx responses to Catch node

☐ Disable strict HTTP parsing

Return a parsed JSON object

Tip: If the JSON parse fails the fetched string is returned as-is.

Headers

+ add

5. Click **Done**, and then connect a debug node at the end.

6. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.

